DAT42-4

Achieving a ML Proof of Concept

Scoping, building, documenting, and defending an end-to-end machine-learning project

---

The capstone: turn everything from S1–S4 into one coherent, reproducible, defensible project.

Albert School — 22 hours

Contents

Preface

This is the capstone of the machine-learning sequence. By now you can do every **piece** of an ML project. In **Machine Learning II** (DAT42-1) you learned to build and compare supervised models — trees, forests, boosting — inside a leak-proof pipeline, and to write an evaluation report that justifies the model you ship. In **Machine Learning III** (DAT42-3) you learned unsupervised methods and, more importantly, that an unsupervised result is only as good as the business framing behind it. In **Exploratory Data Analysis & Data Quality** (DAT32-2) you learned to turn a vague business situation into an answerable question, to profile a dataset honestly, and to document hypotheses before fitting anything. In **APIs, Automation & Data Pipelines** (DAT32-4) you learned to build a re-runnable, idempotent pipeline that survives the three common failure modes. And in **Software Engineering & Code Design** (DAT22-3) you learned the Git, environment, and project-structure discipline that makes any of this reproducible by someone who is not you.

This course teaches nothing new about **models**. It teaches the thing that no single earlier course could: how to **integrate** all of it into one project that a stranger can run, a skeptic can trust, and a mixed audience can understand. That integration is a skill in its own right, and it is the skill that separates a student who "knows scikit-learn" from one who can deliver a **proof of concept** — a working, evaluated, honestly-bounded demonstration that an ML approach is worth pursuing, together with a clear account of what it would take to make it real.

The deliverable is an end-to-end project: a scoped problem, a full EDA, at least two competing models with a justified choice, a reproducible pipeline, a model card, a production-readiness plan, a 10-minute presentation to a mixed audience, and a structured peer review of another team's work. We treat each of these as a craft with its own standards. Throughout, one running example anchors the abstractions: a fictional meal-kit company, **Marmite**, that wants to predict which subscribers will cancel next month so its retention team can intervene.

A note on what a proof of concept **is not**. It is not a production system, not a polished product, and not a research paper. It is the smallest honest experiment that answers one question: **is this problem learnable from the data we have, well enough to be worth building for real?**

Everything in this book serves that question — including, crucially, the parts that teach you to say "not yet, and here is why."

1 What a proof of concept is, and how this project is judged

Before the first line of code, fix the goal. A PoC is judged less on the accuracy number it reaches than on whether the **whole chain of reasoning** holds: did you ask the right question, look honestly at the data, compare real alternatives, make the result reproducible, document the limits, and communicate it to the people who must act on it? A model at 0.91 AUC inside an unreproducible notebook, with no statement of its limitations, is a weaker PoC than a model at 0.84 that a colleague can re-run and a manager can trust.

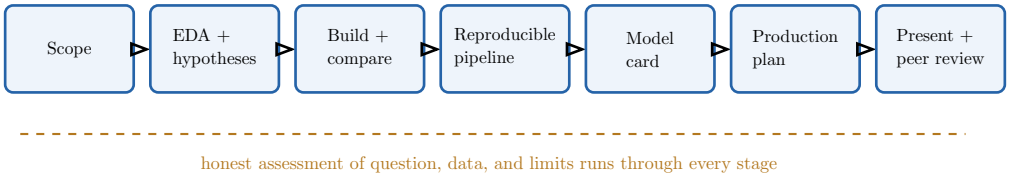


Figure 1: The arc of the capstone project, and of this book. Each stage is a deliverable with its own standard. The dashed thread is the discipline inherited from DAT32-2: every stage states honestly what it does and does not establish. The project is judged on the integrity of the whole chain, not on any single number.

Definition 1 A **proof of concept (PoC)** is the smallest honest experiment that establishes whether a machine-learning approach can solve a defined business problem well enough to justify further investment. It delivers a **scoped problem**, a **working and evaluated model**, a **reproducible pipeline**, **honest documentation of limits and risks**, and a **plan for what would change to reach production**. It is explicitly **not** a production system.

Remark Throughout this course you work in a **team** and present to a **mixed audience**. Both facts shape every deliverable: code must be readable by a teammate (the DAT22-3 discipline), and every claim must be expressible to a non-technical stakeholder (the DAT32-2 discipline). Keep both readers in mind from the first day.

2 Scoping the project: from business problem to project brief

Every weak ML project shares one root cause: it started from "let's build a model" instead of from a question worth answering. You met this discipline in DAT32-2 for analysis; here you extend it to a **modelling** project, where the extra question is **what, exactly, does the model predict?**

Worked example — a vague ask becomes a scoped problem. Marmite's Head of Retention says: "We're losing too many subscribers. Can AI help?" That is not a problem you can build for — it has no target, no data named, no definition of success. Scoping turns it into: **"Predict, for each active subscriber, the probability they will cancel within the next 30 days, so the retention team can offer a discount to the highest-risk 200 customers each week. Success means the flagged group cancels at a materially higher rate than a random group of the same size."** Now there is a target (cancel in 30 days), a user (the retention team), an action (a weekly discount list), and a test of success.

2.1 The four questions of scoping

Definition 2 **Scoping** an ML project answers four questions before any modelling:

1. **The business problem** — what decision or action will the model's output drive, and who acts on it?
2. **The target variable** — the precise quantity the model predicts, including its definition, its time horizon, and how it is measured in the available data.
3. **Data availability** — what data exists, whether it is accessible and of sufficient quality, and — critically — whether it will be available **at prediction time**.
4. **Success criteria** — what "good enough" means, stated as a metric and a threshold tied to the business action, plus what is explicitly **out of scope**.

The output is a short **project brief** recording all four, agreed with the stakeholder.

The single most consequential — and most often botched — choice is the **target variable**. A loose target quietly redefines the whole project.

Common pitfall **Defining the target carelessly is the most expensive mistake in scoping.** "Predict churn" is ambiguous: is a subscriber who pauses for a month "churned"? Is one who downgrades? Over what horizon — 7 days, 30, 90? A model trained on one definition answers a different business question than the one the stakeholder had in mind, and nobody notices until the predictions are useless in production. Pin the target to an exact, measurable event ("subscription status = cancelled, observed within 30 days of the prediction date") and write it in the brief.

Common pitfall **Target leakage hides in the target's timing.** A feature that is only known **after** the outcome it predicts will inflate every score and then vanish in production. If `cancellation_reason` is filled in only once a customer has cancelled, it predicts churn perfectly in your historical data and is **never available** when you actually need a prediction. When scoping the target, ask of every candidate feature: **would I know this value at the moment I must predict?** If not, it cannot be used. This is the leakage you met in DAT42-1, now traced to its source in the problem definition.

2.2 The project brief

Definition 3 A **project brief** is a one-page document, written before modelling and agreed with the stakeholder, stating: the business problem and the decision it informs; the precise target variable and prediction horizon; the data sources and their known limitations; the success metric and threshold; the explicit out-of-scope boundaries; and the intended user of the output. It is the contract the rest of the project is measured against.

Remark The brief is a living contract, not a vow. If the EDA reveals that the data cannot support the chosen target — say, too few cancellations to learn from — you **revise the brief and tell the stakeholder**, rather than quietly drifting to an easier target. Honest renegotiation is part of the method, not a failure of it.

3 Exploratory data analysis that informs the model

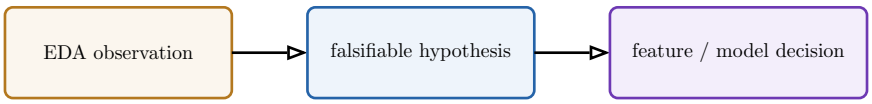
DAT32-2 taught EDA as the disciplined investigation of a dataset; DAT42-1 and DAT42-3 taught you what models need from their inputs. Here the two join: EDA in a modelling project is not just description, it is **reconnaissance for the model**. Every chart should change a decision you will make later — which features to build, which model family to try, which data-quality problem to fix first.

> **Worked example — an EDA finding that redirects the modelling.** Profiling Marmite's data, you find that `tenure_months` is missing for 30% of rows — and that the missingness is not random: it is absent exactly for customers who signed up before a 2024 system migration, who also churn far less. That is **MAR** missingness (DAT32-2), and it carries signal. Naively imputing the median would erase it; instead you add an `is_pre_migration` flag. One EDA finding has just created a feature and prevented a silent bias. This is what "EDA informs the modelling" means.

3.1 Hypotheses as the bridge from EDA to modelling

The OP requires you to document **at least five hypotheses** that inform the modelling approach. A hypothesis is the explicit form of a hunch: a statement about the data that, if true, tells you what to build.

Definition 4 A **modelling hypothesis** is a falsifiable statement, grounded in the EDA, about a relationship in the data and its consequence for the model. A good one names a **variable**, a **claimed relationship**, the **evidence** for it, and the **modelling action** it implies — for example: "Customers who contacted support in the last 30 days churn more (bivariate plot shows 3× the rate); therefore engineer a `recent_support_contact` feature and expect a tree model to find an interaction with tenure."



"support contacts spike before churn" → "recent contact raises risk" → build `recent_support_contact`

Figure 2: A hypothesis is the hinge between looking and building. Each of your five-plus hypotheses should trace this path: an observation in the EDA becomes a falsifiable claim, which dictates a concrete feature-engineering or model-selection decision. Hypotheses with no modelling consequence are decoration; drop them.

Common pitfall **Confusing correlation with causation will mislead your stakeholder, not just your model.** A model is free to exploit a correlation it cannot explain — that is fine for prediction. But the moment you tell the retention team "**contacting support causes churn, so stop answering the phone**," you have made a causal claim the data does not support (recall Simpson's paradox from DAT32-2). State plainly which findings are **predictive** (safe to model on) and which are **causal** (require an experiment to establish). The PoC predicts; it does not prove cause.

4 Building and comparing models

DAT42-1 already taught you to build trees, forests, and boosters and to compare three architectures with a written rationale. The capstone narrows the requirement to **at least two competing architectures with a justified selection**, but raises the bar on the **justification**: the choice must be defensible to both a data scientist and a manager.

4.1 A fair contest, and a meaningful baseline

Definition 5 A **fair model comparison** trains every candidate on the **same** train/validation split, with the **same** preprocessing, evaluated by the **same** metric on the same folds — so differences in score reflect the **models**, not the experimental setup. It always includes a **baseline**: a trivial predictor (majority class, or a simple rule) that any real model must beat to justify its complexity.

Worked example — why the baseline comes first. Before comparing a random forest with a gradient booster on Marmite churn, you fit the baseline: "predict the historical churn rate for everyone." It scores 0.50 AUC by construction (no discrimination). You also fit a one-rule heuristic the business already uses — "flag anyone who hasn't ordered in 21 days" — which scores 0.71 AUC. **That** is the bar. A fancy model at 0.73 is barely worth its complexity; one at 0.86 clearly earns its place. Without the baseline you would not know which world you are in.

4.2 Justifying the selection on more than accuracy

Definition 6 A **justified model selection** records, for each candidate, not only its score on the agreed metric but the factors that bear on **shipping** it: interpretability, training and inference cost, robustness, maintenance burden, and fit to the business constraint. The final choice is the model that best serves the **business problem from the brief**, which is not always the most accurate one.

Candidate	AUC-ROC	Interpretable?	Inference cost
Baseline (21-day rule)	0.71	fully	trivial
Logistic regression	0.83	coefficients	low
Random forest	0.86	importances only	moderate
Gradient boosting	0.88	importances only	moderate

Table 1: A comparison table is the spine of the written rationale. The booster wins on AUC, but if the retention team must **explain** each flag to a customer, logistic regression's signed coefficients may justify a two-point accuracy sacrifice. The right column exists precisely so the decision is not made on the accuracy column alone. State the metric, the scores, **and** the trade-off you chose.

Common pitfall **Picking the highest-accuracy model reflexively is a scoping failure in disguise.** If the brief says the output must be explained to customers, an unexplainable model that scores two points higher does not serve the problem. The selection

rationale must connect back to the brief — accuracy is one input to the decision, not the decision itself.

5 The reproducible pipeline

This is where DAT32-4 and DAT22-3 earn their place in an ML course. A PoC whose result cannot be reproduced is not evidence of anything — it is an anecdote. The OP's standard is exact and demanding: **a different person, on a different machine, must be able to re-run your pipeline from raw data to model output and get the same result.**

Worked example — the reproducibility test, failed and fixed. You hand your teammate the project. They clone the repo and it breaks: a hard-coded path `/Users/you/Desktop/data.csv`; a package version you never pinned; a notebook cell that must be run out of order; a random split with no seed, so the "0.88 AUC" is unrepeatable. Each failure is one you were taught to prevent — `requirements.txt` discipline (DAT22-3), relative paths and a documented entry point (DAT32-4), a fixed `random_state`. Fixed: a `README` with setup steps, a pinned environment, a single `run.py` that ingests raw data and writes the model and metrics, and seeds everywhere. Now the 0.88 is **evidence**.

Definition 7 A **reproducible ML pipeline** takes raw data as input and produces the model and its evaluation as output, as a re-runnable program (not an ad-hoc notebook session), satisfying:

- **isolated, pinned dependencies** — a `requirements.txt` (or lockfile) that reconstructs the environment on another machine (DAT22-3);
- **no hidden state** — every path relative or configured, no manual steps, no reliance on cell execution order; a single documented entry point runs the whole chain;
- **fixed randomness** — seeds set for splits, sampling, and model initialisation so results are bit-for-bit repeatable;
- **idempotency** — re-running on the same input yields the same output without corrupting or duplicating data (DAT32-4);
- **documentation** — a `README` that lets a colleague run it without reading the code.

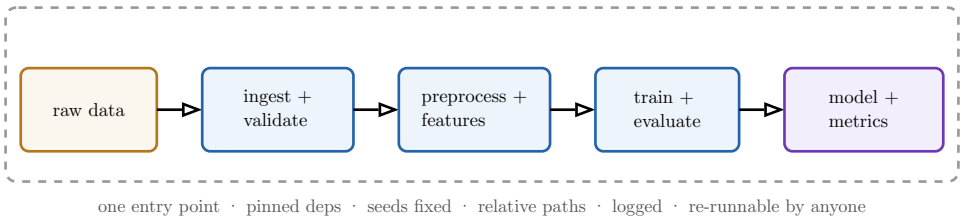


Figure 3: The pipeline as a single re-runnable program, framed by the reproducibility contract (dashed border). The boxes are the familiar ETL + modelling steps; the border is what turns them into a PoC. Anyone who clones the repository and follows the `README` reproduces the `metrics` exactly. The contract is the deliverable, not the boxes.

Common pitfall **The notebook that produced your best number is not a pipeline.** A notebook run top-to-bottom-then-fiddled, with a model fitted in cell 12 using a dataframe mutated in cell 30, cannot be reproduced even by you next week. Promote the finished logic into a script (or a clean, top-to-bottom notebook with no hidden state) before you call the

result reproducible. “It works on my machine” is the exact failure this deliverable exists to prevent.

6 The model card

A working model is a liability until its boundaries are documented. The **model card** is the artefact that records what the model is for, what it was trained on, how well it works, and — most importantly — where it must **not** be trusted. It is the honesty of DAT32-2′s limitations-disclosure, formalised for a model.

Worked example — a limitation that belongs on the card. Marmite’s churn model scores 0.88 AUC overall — but on the EDA you noticed only 1,400 of 40,000 customers are on the annual plan, and the model’s recall on **annual** churners is a dismal 0.4. The card must say so: “Performs well on monthly subscribers; **unreliable for annual subscribers** due to low sample size; do not use to drive annual-plan retention.” Omitting this lets someone deploy the model exactly where it fails. The card’s job is to make that impossible to do by accident.

Definition 8 A **model card** is a concise document accompanying a model, covering:

- **intended use** — the problem it solves, the user, and explicitly **out-of-scope** uses;
- **training data** — its source, time range, size, and known biases or gaps;
- **evaluation** — the metric(s), the score(s), and performance **broken down across meaningful subgroups**, not just the aggregate;
- **limitations** — where the model is unreliable and why;
- **risks** — the harms of being wrong (false positives and false negatives have different costs), and any fairness or ethical concerns.

Common pitfall **A single aggregate metric hides the failures that matter.** “0.88 AUC” can mask a subgroup where the model is no better than chance (the annual-plan customers above). Always report performance **disaggregated** across the segments your stakeholder cares about. A model card that reports only the headline number is not being honest; it is being incomplete in the direction that flatters the model.

Remark The model card connects directly to the brief. Its “intended use” is the brief’s business problem; its “evaluation” is the brief’s success criterion, measured. Reading the brief and the card side by side should tell a stranger the entire story of the project’s intent and its honest result.

7 From proof of concept to production

A PoC’s final intellectual act is to chart the road it is **not** taking yet. The OP asks for **the three most important next steps** to move toward production, and an explanation of **what would change at each**. This is a prioritisation exercise: not an exhaustive checklist, but the three changes that matter most for **this** project.

Worked example — three next steps for Marmite. The churn PoC trains once, on a static CSV, evaluated offline. The three steps that matter most to make it real:

1. **Data freshness and pipeline automation** — production needs the prediction to run automatically every week on **live** data (the DAT32-4 scheduling and idempotency skills), not on a one-off export.
2. **Monitoring for drift** — subscriber behaviour shifts; the model must be watched for degrading performance and retrained on a schedule, with alerts when the flagged group stops cancelling at an elevated rate.
3. **Serving and integration** — the weekly risk list must reach the retention team's actual tool, with latency, access control, and a fallback when the model is unavailable.

Each names **what changes** from the PoC: batch-to-scheduled, train-once-to-monitored, notebook-output-to-integrated-service.

Definition 9 A **production-readiness plan** identifies the **three highest-priority** changes required to move the PoC toward a deployed system and states, for each, **what concretely changes** relative to the PoC. Common axes are: **data** (one-off export → automated, monitored ingestion), **model lifecycle** (train-once → monitored, retrained, versioned), and **serving** (offline output → integrated service with latency, access, and failure handling). The plan is a prioritisation, so it justifies **why these three** over the alternatives.

Common pitfall **"Just deploy the notebook" skips every reason the PoC was only a proof.** The PoC deliberately cut corners — static data, offline evaluation, no monitoring — to answer the learnability question cheaply. The production plan is where you name those corners as the work that remains. A plan that ignores data freshness, monitoring, or failure handling is not a plan; it is an assumption that the PoC's shortcuts are free, and they are not.

8 Communicating the project

A PoC that no one understands persuades no one. The OP requires a **10-minute presentation to a mixed technical and non-technical audience**, with questions from both sides. This is the DAT32-2 communication discipline under a harder constraint: two audiences, one talk, ten minutes.

8.1 Structuring a talk for two audiences at once

Definition 10 A **mixed-audience presentation** leads with the **business problem and the result** (what every listener can follow), supports it with **just enough method** to make the result credible, and is honest about **limitations**. It is structured so a non-technical listener never gets lost and a technical listener never feels talked down to — typically by keeping the main thread non-technical and parking depth in reserve for questions.

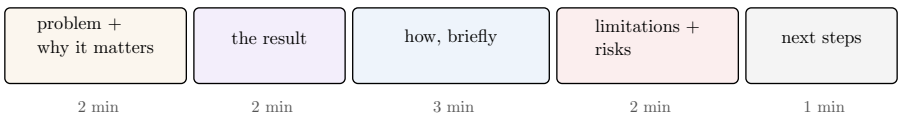


Figure 4: A defensible shape for the 10-minute talk. The business problem and the result come first and get the most accessible language; method is compressed to what makes the result believable; limitations and next steps close honestly. The technical depth lives in your back pocket for the Q&A, not in the main thread.

Common pitfall **Opening with your methodology loses the room in the first minute.** Starting "we used a gradient-boosted ensemble with out-of-fold target encoding…" tells the non-technical half nothing about **why they should care**, and you never get them back. Lead with the problem and the result in plain language; bring the machinery out only when a technical question pulls for it.

Remark **Fielding questions from both sides** is a graded skill. A non-technical question ("can we trust this?") wants a plain-language answer about limitations and risk; a technical question ("how did you handle leakage?") wants specifics. Diagnose which kind you are getting **before** you answer, and resist giving a technical answer to a business question or vice versa.

9 Peer review

The course closes by turning you into the evaluator. The OP requires a **written peer review of another team's project using a structured rubric**. Reviewing is how you internalise the standards — it is far easier to see a missing limitations section in someone else's model card than in your own.

Definition 11 A **structured peer review** evaluates another project against a **fixed rubric** applied to every project alike, scoring each dimension of the work — scoping, EDA and hypotheses, modelling and justification, reproducibility, the model card, the production plan, and communication. Each judgement is **specific and evidence-based** ("the model card reports no subgroup performance"), **actionable** ("disaggregate AUC by subscription plan"), and **constructive** in tone. The rubric makes reviews comparable and fair across teams.

Worked example — vague feedback versus a useful review. Weak: "The model section was confusing and the results seem a bit weak." This cannot be acted on. Strong: "The comparison omits a baseline, so I cannot tell whether 0.86 AUC beats the existing 21-day rule (scoping rubric, −1). Suggest adding the trivial and heuristic baselines from the brief." Same concern, but the second cites the rubric, points to specific evidence, and proposes a fix. That is the difference the structured rubric is designed to produce.

Common pitfall **A reproducibility claim you did not test is not a review.** The single highest-value act of peer review is to **actually clone and run** the other team's pipeline on your machine. If it does not reproduce, that is the most important finding you can report — and the one the authors are least able to see themselves, because it runs fine on the machine where it was built.

The arc of this course. A proof of concept is the smallest honest experiment that shows whether an ML approach is worth building, and it is judged on the integrity of the whole chain, not one accuracy number. **Scope** first: a precise business problem, an exactly-defined target (the leakage-prone choice), assessed data, and a written brief. **Explore** to inform the model, turning EDA findings into at least five falsifiable hypotheses that each dictate a feature

or model decision. **Build and compare** at least two architectures against a meaningful baseline on a fair setup, and justify the choice on more than accuracy. Make the whole thing a **reproducible pipeline** a stranger can re-run — the DAT22-3 and DAT32-4 discipline made non-negotiable. Document the model in a **model card** that discloses limitations and disaggregated performance, plan the **three highest-priority steps to production**, and **present and peer-review** with honesty about what the PoC does and does not establish. The skill this course certifies is integration: turning every piece you already own into one coherent, defensible project.

Exercises

1. Take a vague business ask ("can we use AI to reduce returns?") and write a one-page project brief: the business problem and the decision it drives, the precise target variable with its time horizon, the data sources and their limitations, the success metric and threshold, and the explicit out-of-scope boundaries. Identify one candidate feature that would constitute target leakage and explain why.

2. Conduct a full EDA on your project dataset and document **at least five** hypotheses. For each, state the EDA evidence, the falsifiable claim, and the concrete feature-engineering or model-selection decision it implies. Mark each finding as predictive or causal and justify the label.

3. Fit a trivial baseline and a simple heuristic baseline for your problem before any real model. Report their scores on your chosen metric, and explain what bar a real model must clear to justify its complexity.

4. Build and evaluate **at least two** competing architectures on an identical train/validation setup. Produce a comparison table including a non-accuracy column (interpretability, inference cost, or maintenance), and write a rationale that selects a final model by reference to the project brief — not to accuracy alone.

5. Make your pipeline reproducible: pin dependencies, remove hard-coded paths and hidden state, fix all random seeds, provide a single documented entry point, and write a `README`. Then have a teammate clone it on a different machine and reproduce your headline metric. Report any failure and how you fixed it.

6. Write a model card for your final model covering intended use (including out-of-scope uses), training data and its biases, evaluation with performance **disaggregated across at least one meaningful subgroup**, limitations, and the risks of false positives versus false negatives.

7. Write a production-readiness plan naming the **three** highest-priority steps to move your PoC toward production, stating concretely what changes at each and justifying why these three over the alternatives.

8. Prepare and deliver a 10-minute presentation of your project to a mixed audience. Structure it to lead with problem and result, compress the method, and close on limitations and next steps. Afterwards, write down one business question and one technical question you received and how you tailored each answer.

9. Conduct a structured peer review of another team's project against a fixed rubric covering every dimension of the work. Make each judgement specific, evidence-based, and actionable, and include the result of actually re-running their pipeline on your machine.